

Arquiteturas de Alto Desempenho 2026/2027

Practical work guide AAD_P3 (2026-09-28)

Tomás Oliveira e Silva

1 Objectives

Learn how to parallelize code the easy way (OpenMP), with good, but not total, control of what is going on. List of things you will learn about:

- how to parallelize a section of code (possibly with one or more critical sections),
- how to parallelize a for cycle,
- how to reduce a for cycle (conceptually similar to a MapReduce), and
- how to use tasks to parallelize things.

This is only the tip of a large iceberg. Some OpenMP clauses are mentioned in this document without explaining them. If you want to learn more, and there is much more to learn, consult the OpenMP documentation.

2 Running a code section in parallel

To parallelize a piece of code using pthreads, we had to write down a function, place that piece of code inside it, add glue code to pass parameters and return values, and call `pthread_create` and `pthread_join` to launch and regroup the threads. OpenMP does that in a much simplified way. It is only necessary to annotate the source code, and the compiler does the rest of the work (creating temporary variables for input/output data, thread management, and so on). The annotation is done using so called pragmas. For example, to run a section of code in parallel, using 8 threads, one just has to write

```
# pragma omp parallel num_threads(8)
{
    int thread_number = omp_get_thread_num();
    // rest of the code (the work to do may depend on the thread number)
}
```

and that's it! All automatic variables (without `static` in their declarations) declared inside the section of code will not be shared between the threads. Variables declared outside of the loop are shared. It is possible to explicitly specify that some variables are private to the thread (`private` clause) or shared between threads (`shared` clause). In the later case, be aware of race conditions if you change them!

Inside the section of code we can use the `omp_get_thread_num()` function to find out the thread number, and that can be used to specify the work that each thread will be doing. The end of the block is a barrier; the program only advances after all threads have reached it.

It is also possible to parallelize a for loop. The annotation is quite similar. Here's an example:

```
# pragma omp parallel for num_threads(8)
for(i = 0; i < 1000000; i++)
{
    // loop code
    // iteration i MUST not depend of earlier iterations
    // but see the reduction section later in this document
}
```

The compiler splits the loop range in smaller ranges and each thread does its own range. Nice!

It is also possible to parallelize nested for loops if the range of each inner for loop does not depend on the variables used in the outer for loops. Just use the collapse clause (look it up in the manual!)

The file `create_and_join.c`, which has the same name as one of the previous practical class, but which has different code, puts these two annotations in action. Compare its code with the one of the previous class:

```
$ meld create_and_join.c ../P2/create_and_join.c
$ # or
$ vi -d create_and_join.c ../P2/create_and_join.c
```

They do essentially the same thing, one with pthreads, the other with OpenMP pragma directives. Which one is simpler?

Compile and run it:

```
$ make create_and_join
$ ./create_and_join
```

Notice how the for loop indices are split among the threads.

It is now time to roll up our sleeves and take a look at the code the compiler generates using OpenMP pragmas. This can be done using

```
$ objdump --disassemble ./create_and_join | less
```

or compiling the code to assembly (this is already done for you in the makefile):

```
$ make create_and_join.s
```

If the latter, open the file `create_and_join.s` with a text editor and examine its contents. Besides the code of the `main()` function, do you see code for other functions?

3 Critical sections

It is also possible to put critical sections inside a parallel code section:

```
# pragma omp parallel
{
    // bla, bla, bla
    # pragma omp critical
    {
        // critical section
    }
    // bla, bla, bla
}
```

The compiler will automatically create and use a mutex to control accesses to the critical section. It is possible to give a name to the critical section: just append it at the end of the pragma line, inside parenthesis:

```
# pragma omp critical (your_name_of_the_critical_section)
```

If two or more critical sections share the same name, they will use the same mutex.

The file `lock_and_unlock.c`, which has the same name as one of the previous practical class, but which has different code, puts this annotation in action. Compare its code with the one of the previous class. Which one is simpler?

Compile and run it:

```
$ make lock_and_unlock
$ ./lock_and_unlock
```

It does the two cases — with and without a critical section – in one go. Is its behavior similar to that of the code of the previous class?

Add code, perhaps using the `wall_time()` function declared in the `create_and_join.c` file, to measure the time it takes to execute the parallel code with, and without, the critical section. How do execution times compare with those of the code of the previous class?

If some cases using an atomic clause (do to an atomic update of a shared variable) is more efficient than using a critical section (with its `critical` clause). However, a critical section is more general. It must be used, for example, when it is desired to update more than one shared variable in a safe way.

4 Reduction

Sometimes we use a for loop to sum the elements of an array (or, for example, to get their minimum or maximum). OpenMP has a way to do this, minimizing the amount of code we have to write. One just appends `reduction(op:var_list)` and the end of a `parallel` pragma. For example, to compute the sum of the elements of an array in parallel, we can do

```
int sum = 0;
# pragma omp parallel for reduction(+:sum)
for(int idx = 0; idx < array_size; idx++)
    sum += array[idx];
```

The compiler will compute partial sums in each thread, using a local variable and without mutexes, and at the end the values of the local variables are “reduced” (now with a mutex or, possibly, an atomic operation) using the specified reduction function (in this case, `+`). All variables specified after `:` are treated in this way.

To see this in action, compile and run the code provided in `reduction.c`:

```
$ make reduction
$ ./reduction
```

As usual, you should take a good look at the source code. Modify the code so that the reduction returns the minimum and maximum values of the array.

5 Tasks

It is very easy to create a pool of worker threads in OpenMP. For that, use the `task` clause. You can look up the details about how this is done in the OpenMP manual ([clickable link](https://www.openmp.org/specifications/)).

URL: <https://www.openmp.org/specifications/>

File `tasks.c` presents a simple example of using them. Just examine it (read carefully the comments, they explain what is going on), compile, and then run it.

After doing all this, cleanup:

```
$ make clean
```

6 Putting it all together: parallelization of `MandelbrotCount.c`

It is now time to put all that you learned about OpenMP in action. Modify the `MandelbrotCount.c` source code so that it runs in parallel. Which is easier: doing it with `pthread`s or using OpenMP?